

HygieneBench: A Reproducible Synthetic Benchmark for Cyber-Hygiene Anomaly Detection Across Identity, Endpoint, and Patch Telemetry

Harshavardhan Malla Independent Researcher

Abstract:

Cyber-hygiene anomaly detection, identifying stale privileged accounts, dormant account reactivations, endpoint coverage gaps, patch noncompliance clusters, and telemetry missingness, is operationally important but poorly served by existing public benchmarks, which focus predominantly on network intrusion or attack-emulation telemetry. We introduce **HYGIENEBENCH**, an open, reproducible benchmark that jointly covers Active Directory identity state, endpoint patch posture, vulnerability exposure, and telemetry freshness across seven evaluation tasks (T1-T7) and twelve anomaly classes. **HYGIENEBENCH** is built entirely from synthetic, seeded telemetry generated from publicly citable structural priors (NIST NVD severity distributions, Verizon DBIR 2026 patch-lag aggregates [1], CISA BOD 23-01 telemetry-cadence requirements), with no employer or production data at any stage.

We evaluate eight methods, a task-specific rule baseline (M1), a hybrid risk scorer (M2), Isolation Forest (M3), Local Outlier Factor (M4), One-Class SVM (M5), a linear autoencoder (M6), population-level temporal z-score (M7), and a graph-augmented Isolation Forest (M8), across five telemetry conditions (baseline, fresh, stale, missing, unsupervised). Applying a pre-registered failure protocol ($\Delta < 0.05$ AP and $\Delta < 0.05$ P@k in $\geq 2/3$ seeds), we find that 83.8% of (condition, task, method) configurations fail to improve on the rule baseline, with ML adding meaningful signal on T2 (group-membership-drift detection, best ML: M7 temporal z-score, +0.185 AP over rule) and T5 (patch-vulnerability hygiene, best ML: M5 OCSVM, +0.210 AP over rule). Telemetry staleness (C-STALE) consistently degrades detection performance relative to the baseline condition. We release the generator, datasets, and evaluation harness to enable reproducible comparison of future methods.

Index Terms: cyber hygiene, anomaly detection, benchmark, synthetic data, identity security, endpoint management, negative-result reporting

I. INTRODUCTION

Maintaining cyber hygiene, keeping privileged accounts active and monitored, endpoint agents deployed, patches current, and telemetry fresh, is a persistent operational challenge for security operations centers (SOCs), particularly in public-sector environments subject to compliance mandates such as CISA Binding Operational Directives [2] and NIST SP 800-40 [3]. Failures in hygiene posture create the enabling conditions for identity-based attacks: dormant accounts reactivated by adversaries, endpoints missing agent coverage, and stale privileged credentials that may be invisible to behavioral analytics requiring recent baselines.

Despite the operational importance of hygiene-state monitoring, the anomaly-detection research community lacks a dedicated public benchmark for this problem. Existing benchmarks, the LANL Unified Host and Network Dataset [4], CERT Insider Threat datasets [5], CICIDS [6], and attack-emulation repositories such as Mordor/Security Datasets [7], provide rich telemetry for network intrusion or insider-threat detection but do not model identity hygiene state, patch posture, vulnerability exposure, or telemetry freshness as first-class evaluation axes. Commercial platforms (Microsoft Defender for Identity, Splunk UBA, Exabeam) address related detection problems but are closed and do not release benchmark datasets.

This gap matters for two reasons. First, researchers who develop anomaly-detection methods for security operations have no way to compare their methods on hygiene-specific telemetry under controlled conditions. Second, there is no principled accounting of when unsupervised ML methods add value over simpler rule-based approaches for hygiene anomalies, a question with real operational implications, since rule maintenance is expensive and ML deployment requires justification.

This paper makes the following contributions:

- **C1: Synthetic generator.** A fully open, seeded synthetic telemetry generator covering eleven entity/event tables (Active Directory users, groups, computers, assets; login events; group membership events; endpoint patch state; vulnerability records; remediation events; account lifecycle events; telemetry freshness log) with documented structural priors.
- **C2: Anomaly taxonomy.** Twelve cyber-hygiene anomaly classes (AH-01 through AH-12) with ATT&CK enabling-condition mappings (not technique detection claims), injection protocols, and class imbalance specifications.
- **C3: Benchmark task suite.** Seven evaluation tasks (T1-T7) under five telemetry conditions, with pre-registered k values, feature sets, and method applicability flags.
- **C4: Comparative evaluation, failure-aware.** An 810-run evaluation of eight methods across all tasks and conditions, with a pre-registered failure protocol reporting when ML does not consistently improve on the rule baseline.
- **C5: Negative-result reporting.** Explicit reporting and analysis of the 83.8% of configurations where ML fails to beat the rule baseline by the pre-registered threshold, with discussion of which task and condition properties drive this

outcome.

The remainder of the paper is organized as follows. Section II reviews related benchmarks. Section III describes the HYGIENE BENCH design. Section IV presents experimental setup. Section V reports results. Section VI discusses findings and limitations. Section VII concludes.

II. RELATED WORK

a) Network intrusion and attack emulation benchmarks.: The LANL Unified Host and Network Dataset [4] provides large-scale enterprise network flow and authentication telemetry, but it captures network-level events and does not model identity hygiene state (account staleness, privilege drift) or endpoint patch posture. CICIDS [6] and DARPA datasets [8] focus on network traffic classification. Mor-dor/Security Datasets [7] and BOTSv3 [9] provide attack-emulation telemetry from red-team exercises, a fundamentally different framing from hygiene-state monitoring. None of these benchmarks includes telemetry freshness or missingness as a controlled evaluation variable.

b) Insider threat datasets.: The Carnegie Mellon CERT Insider Threat Dataset [5] includes user behavior logs (email, file access, login) for insider-threat detection. While it covers some identity-adjacent behaviors, it does not model endpoint patch state, vulnerability records, or the multi-table hygiene-state structure of enterprise AD environments.

c) Graph anomaly detection.: The graph anomaly detection literature has produced several methods and datasets for attributed-graph anomalies [10], [11]. These approaches typically operate on single-hop graph structure and do not model the temporal, multi-table telemetry characteristic of enterprise security operations.

d) Commercial and proprietary platforms.: Microsoft Defender for Identity, Entra ID Protection, Splunk UBA, Exabeam, and Microsoft Sentinel address overlapping detection problems. They are commercial products, not open benchmarks, and do not release datasets or evaluation methodologies for external comparison. We make no comparison claims against these platforms.

e) Synthetic security dataset generation.: Synthetic generation of security telemetry has precedent in network simulation [12]. HYGIENE BENCH extends this tradition to the hygiene-state domain with documented structural priors and a reproducible seeded generator.

f) Failure-aware and negative-result reporting.: The practice of explicitly reporting when ML fails to outperform simpler baselines is well-established in machine learning [13] but underrepresented in security benchmark papers. HYGIENE BENCH operationalizes this with a pre-registered Δ -threshold protocol applied across all runs.

III. HYGIENE BENCH DESIGN

A. Schema and Synthetic Generator

HYGIENE BENCH v0.1 uses a schema with eleven entity/event tables and one anomaly label table (Table I). The generator (SyntheticHygieneGenerator) is implemented in Python using NumPy and Pandas, with all randomness gated through

TABLE I
HYGIENE BENCH v0.1 SCHEMA (MEDIUM SCALE, $n=1,000$ USERS).

Table	Rows	Description
users	1,000	AD user accounts with identity state
groups	65	AD security/distribution groups
computers	830	Managed endpoints
assets	830	Asset inventory entries
login_events	~249k	Auth. events (30-day window)
group_membership_events	~51	Group add/remove events
account_lifecycle_events	varies	Account create/disable/enable
endpoint_patch_state	830	Per-endpoint patch compliance
vulnerability_records	~3.5k	Per-endpoint CVE exposure
remediation_events	varies	Patch and remediation actions
telemetry_freshness_log	varies	Per-entity data-source freshness
anomaly_labels	110	Ground-truth labels (task+split)

`numpy.random.default_rng(seed)`. Entity identifiers are deterministic UUIDs derived from `hashlib.md5(seed_tag + index)`, ensuring reproducibility across platforms.

a) Structural priors.: Patch-lag distributions use DBIR 2026 aggregate statistics [1] (43-day mean for critical patches). CVE severity distributions use NVD base score statistics. Telemetry refresh cadence requirements follow CISA BOD 23-01 (14-day asset discovery, 72-hour patch data) [2]. AD group-size distributions lack a peer-reviewed published source and are modeled via disclosed assumptions (power-law approximation) with sensitivity sweeps. All structural priors are documented in the dataset card.

b) Conditions.: Five telemetry conditions parameterize freshness and missingness:

- **C-BASE:** No artificial staleness; all sources current.
- **C-FRESH:** Elevated freshness flagging; sources verified current.
- **C-STALE:** 20% of entities have heavy-stale telemetry (>14-day gap).
- **C-MISS:** One data source absent for 15% of entities.
- **C-UNSUP:** C-BASE with anomaly labels withheld from the training pipeline.

Each condition is generated for three seeds (42, 137, 2024), producing 15 dataset instances (5 conditions $\times$ 3 seeds) with $n=1,000$ users per instance.

B. Anomaly Taxonomy

HYGIENE BENCH defines twelve anomaly classes (Table II), each mapped to an ATT&CK enabling condition (not a detection claim for any specific technique). Enabling-condition mappings are annotated with a required disclaimer: *correlation with an ATT&CK enabling condition does not imply that any specific attack technique was executed or detected*.

TABLE II
HYGIENE BENCH ANOMALY CLASSES. ATT&CK MAPPINGS ARE
ENABLING-CONDITION CORRELATIONS; THEY DO NOT IMPLY DETECTION OF ANY
SPECIFIC TECHNIQUE.

Class	Code	ATT&CK enabling condition	Tasks
Stale privileged account	AH-01	Valid Accounts: Domain (T1078.002)	T1
Privilege escalation drift	AH-02	Valid Accounts; Domain Policy Mod.	T7
Group membership drift	AH-03	Domain Policy Modification (T1484)	T2
Dormant account reactivation	AH-04	Valid Accounts: Domain (T1078.002)	T6
Impossible/unusual login	AH-05	Valid Accounts; Lateral Movement	T1,T3
Endpoint-identity risk corr.	AH-06	Exploit Public-Facing App	T3
Patch noncompliance cluster	AH-07	Exploit Public-Facing App (T1190)	T5
KEV exposure aging	AH-08	Exploit Public-Facing App (T1190)	T5
Asset inventory mismatch	AH-09	Defense Evasion enabling	T4
Missing endpoint agent	AH-10	Defense Evasion enabling	T4
Telemetry missingness cluster	AH-11	Defense Evasion enabling	T4
Abnormal remediation delay	AH-12	Exploit Public-Facing App	T5

C. Benchmark Tasks

Seven tasks cover the twelve anomaly classes across different entity scopes and temporal windows (Table III). Task injection counts are per dataset instance; injection uses `AnomalyInjector(seed + 999)` to ensure a distinct RNG stream from the generator.

a) *Dataset splits.*: Labels are stratified 60/20/20 by anomaly class. Anomalous entities are assigned to the test split, with normal entities split randomly across train/val/test. The split is deterministic given the seed.

IV. EXPERIMENTAL SETUP

A. Methods

We evaluate eight methods representing the main algorithmic families applicable to unsupervised tabular anomaly detection under class imbalance (Table IV).

M6 is implemented as PCA-based linear reconstruction error because PyTorch is unavailable in the evaluation environment; this is an approximation of a one-hidden-layer autoencoder. M8 is implemented using `networkx` graph features (degree, privileged-degree, clustering coefficient) combined with Isolation Forest rather than DOMINANT-style deep graph autoencoders [10], for the same reason. Both approximations are documented. M8 is excluded from T4 and T5 by design (no meaningful graph signal for asset/patch anomalies; reported as N/A).

B. Feature Engineering

For each task, `TaskFeatureExtractor.extract(task_id, seed)` computes a feature matrix from raw CSVs. Features include task-relevant identity state attributes, temporal aggregates (e.g., 30-day login frequency, off-hours rate, cross-segment rate), patch compliance scores, vulnerability counts, KEV exposure counts, and telemetry freshness scores. Missing values are imputed with training-set column means before fitting.

C. Metrics and Negative-Result Protocol

a) *Primary metrics.*: Average Precision (AP) and Precision@ k ($P@k$) at task-specific k values (Table III). AP is the standard interpolated area under the precision-recall curve. $P@k$ is the fraction of true anomalies in the top- k ranked entities, the operationally relevant metric for SOC review budgets.

b) *Secondary metrics.*: Recall@ k ($R@k$), False Positive Burden (FPB = $1 - P@k$), and AP stability ($1 - \text{CV}$ across seeds).

c) *Pre-registered failure protocol.*: A (condition, task, method) configuration is flagged as a *negative result* (►) if:

$$\begin{aligned} \text{AP}(\text{method}) - \text{AP}(\text{M1}) &< 0.05 \\ \text{and } P@k(\text{method}) - P@k(\text{M1}) &< 0.05 \end{aligned} \quad (1)$$

in at least $\lceil 2/3 \times n_{\text{seeds}} \rceil = 2$ of 3 seeds. Flagged configurations are reported in full and are not filtered. This protocol was pre-registered prior to examining any results.

V. RESULTS

A. Overall Performance

Table V summarizes mean AP and $P@k$ for each method, averaged across all conditions, tasks, and seeds (810 runs). Figure 2 breaks the same runs down by method and task. The rule baseline (M1) achieves the highest mean AP (0.916). M5 (OCSVM) is the closest ML competitor ($\text{AP}=0.913$, $P@k=0.447$ vs. M1 $P@k=0.444$). M2 (hybrid scorer) performs worst among all methods ($\text{AP}=0.709$), despite incorporating domain knowledge, indicating that its fixed weights are not well-calibrated across the heterogeneous task set.

Overall: applying the pre-registered failure rule of Eq. (1), 197 of 235 (condition, task, method) configurations are failure-flagged (83.8% negative-result rate). For synthetic cyber-hygiene telemetry with structured, rule-exploitable anomaly injection, simple threshold rules perform at least as well as unsupervised ML methods on the majority of evaluation configurations. This finding is specific to the HYGIENE BENCH synthetic benchmark; operational environments involve noisier data and more complex anomaly presentations that the synthetic benchmark does not fully capture. An earlier pipeline version aggregated failure flags per (condition, seed) instance; all numbers reported here use the pre-registered per-condition aggregation across the three seeds.

TABLE III
BENCHMARK TASKS. k VALUES ARE PRE-REGISTERED PER TASK_SPECS.MD AND REFLECT OPERATIONAL SOC REVIEW BUDGETS.

Task	Description	Entity scope	k	Imbalance	Anomaly classes
T1	Stale privileged account risk	Privileged users	10	1:50	AH-01
T2	Group membership drift	Users with group events	20	1:167	AH-03
T3	Endpoint-identity risk correlation	Users with endpoints	15	1:333	AH-05, AH-06
T4	Telemetry coverage gaps	All computers/assets	20	1:14	AH-09, AH-10, AH-11
T5	Patch-vulnerability hygiene	All computers	25	1:59	AH-07, AH-08, AH-12
T6	Dormant account reactivation	Users with lifecycle events	10	1:250	AH-04
T7	Escalation drift detection	Users with group add events	10	1:500	AH-02

TABLE IV
EVALUATION METHODS. M8 IS EXCLUDED FROM T4 AND T5 (NO GRAPH SIGNAL; REPORTED AS N/A).

ID	Name	Type	Notes
M1	Rule baseline	Task-specific rules	No training; per-task thresholds
M2	Hybrid scorer	Weighted combination	Identity + endpoint + vuln + freshness
M3	Isolation Forest	Ensemble	200 trees, contam.=0.02
M4	LOF	Density-based	$k=20$ neighbors, novelty mode
M5	OCSVM	Kernel-based	$\nu=0.05$, RBF kernel
M6	Linear-AE	Reconstruction error	PCA (latent dim=8); approx. neural AE
M7	Temporal z-score	Statistical	Population z-score, temporal features
M8	Graph-IF	Graph + ensemble	Bipartite features + IF; approx. DOMINANT

TABLE V
MEAN AP AND $P@k$ BY METHOD ACROSS ALL 810 RUNS. ► = FAILURE-FLAGGED (ML DOES NOT IMPROVE ON RULE BY $\Delta=0.05$ IN $\geq 2/3$ SEEDS). †M8 COVERS T1-T3, T6-T7 ONLY ($n=25$ CONFIGS).

Method		Mean AP	Mean $P@k$	Failure rate
M1	Rule baseline	0.916	0.444	
M5	OCSVM	0.913	0.447	68.6% ►
M8	Graph-IF†	0.801	0.348	100% ►
M4	LOF	0.800	0.445	71.4% ►
M6	Linear-AE	0.798	0.407	85.7% ►
M3	Isolation Forest	0.781	0.428	85.7% ►
M7	Temporal z-score	0.774	0.392	85.7% ►
M2	Hybrid scorer	0.709	0.417	94.3% ►
Overall failure rate: 197/235 = 83.8%				

a) *AP stability.*: Per the pre-registered secondary metrics, AP stability ($1 - CV$ across the three seeds, computed per (condition, task, method) group) is high for all methods: mean $1 - CV$ ranges from 0.859 (M8) to 0.967 (M1), with M5 at 0.951 and M2-M7 between 0.879 and 0.951. Full per-configuration stability values, along with the remaining secondary metrics $R@k$ and FPB, are included in the released evaluation artifact.

B. Where ML Adds Value

Despite the dominant negative-result finding, two tasks show consistent ML improvement over the rule baseline (Table VI; Figure 3 shows the full per-method AP comparison on both tasks).

TABLE VI
TASKS WHERE ML CONSISTENTLY ADDS VALUE, I.E., CONFIGURATIONS NOT FLAGGED BY EQ. (1): $\Delta AP \geq 0.05$ OR $\Delta P@k \geq 0.05$ IN $\geq 2/3$ SEEDS.

Task	Best ML	Rule AP	ML AP	ΔAP	Condition
T2	M7 (Z-score)	0.766	0.951	+0.185	C-BASE/STALE
T2	M4 (LOF)	0.855	0.975	+0.120	C-MISS
T5	M5 (OCSVM)	0.458	0.668	+0.210	C-BASE/STALE
T5	M5 (OCSVM)	0.726	0.837	+0.111	C-MISS

a) *T2, Group Membership Drift.*: The rule baseline (M1) achieves mean AP = 0.766 on C-BASE and C-STALE. The best ML method is M7 (temporal z-score), achieving AP = 0.951 (+0.185 AP). M5 (OCSVM) is second at AP = 0.913 (+0.147) and M4 (LOF) third at AP = 0.890 (+0.124). Three ML methods consistently improve on the rule baseline. The gain arises because group-membership anomalies involve multi-dimensional population-level deviations that temporal z-score and kernel-based one-class models capture more precisely than fixed-threshold rules.

b) *T5, Patch-Vulnerability Hygiene.*: This is the hardest task overall (mean AP = 0.617 across all methods). The rule baseline achieves AP = 0.458 on C-BASE/C-STALE, while the best ML method achieves AP = 0.668 (+0.210 AP) via M5 (OCSVM). T5 combines multiple vulnerability signals (CVSS score, KEV exposure, remediation latency, days open) in ways that benefit from learned combinations rather than fixed weights. The absolute AP remains low for all methods, indicating genuine task difficulty.

C. Where Rules Dominate

Five of seven tasks are dominated by the rule baseline under most conditions (Table VII): T1 (stale privileged accounts), T3 (endpoint-identity correlation), T4 (telemetry coverage gaps), T6 (dormant reactivation), and T7 (escalation drift). For these tasks under C-BASE, both M1 and the best ML method achieve AP ≈ 1.000 .

T3 is a partial exception. Under C-BASE its AP is 0.782, but under C-STALE it degrades sharply to 0.613 ($\Delta = -0.169$), the largest condition sensitivity of any task. T3 combines identity-side and endpoint-side features, making it susceptible to staleness in either domain.

T7 is rule-dominated despite extreme rarity: with only 2 anomalous entities per dataset instance (imbalance $\approx 1:500$), the rule-based methods M1 and M7 achieve AP = 1.000, while several ML methods rank the two rare anomalies poorly, M2 has mean T7 AP = 0.319, M5 0.786, M4 0.830, and M3/M8 0.861 (task mean 0.822). No ML method closes the gap to the rule baseline.

TABLE VII
RULE-DOMINATED TASKS (RULE AP $\geq$ BEST-ML AP IN MAJORITY OF CONFIGS).

Task	Mean AP	Notes
T1 Stale priv. accounts	0.937	AP $\approx$ 1.0 under all conditions
T3 Endpoint-identity	0.736	C-STALE sensitivity: -0.169
T4 Telemetry gaps	0.890	High AP; 1:14 imbalance
T6 Dormant reactivation	0.832	AP $\approx$ 1.0; M8 exception (0.490)
T7 Escalation drift	0.822	Extreme imbalance; rules at AP = 1.0

TABLE VIII
MEAN AP BY TELEMETRY CONDITION (ALL METHODS AND TASKS).

Condition	Mean AP	Δ vs BASE	Primary driver
C-FRESH	0.856	+0.084	T5 +0.238; T1 +0.101
C-MISS	0.845	+0.073	T5, T1 dominate
C-UNSUP	0.844	+0.072	Instance-level sampling variation
C-BASE	0.772		Reference
C-STALE	0.741	-0.031	T3 -0.169; staleness

D. Effect of Telemetry Conditions

Table VIII shows mean AP by condition, and Figure 4 shows the corresponding AP distributions. C-STALE produces the lowest mean AP (0.741 vs. C-BASE 0.772), confirming that staleness degrades detection.

The C-FRESH, C-MISS, and C-UNSUP conditions show higher mean AP than C-BASE (0.856, 0.845, 0.844). The largest contributor is T5 (+0.238 AP under C-FRESH). Fresh telemetry improves patch compliance scores, KEV exposure counts, and remediation latency reliability, making the underlying feature signals more discriminating. This is a data-quality effect, not a freshness-flag artifact. The C-UNSUP gap deserves an honest caveat: all eight methods are unsupervised, so withholding labels cannot raise AP. The C-UNSUP/C-BASE difference instead reflects independent RNG draws across separately generated dataset instances. Consequently, between-condition gaps of similar magnitude, including part of the C-FRESH effect, should be read with instance-level sampling variation in mind.

E. Failure-Flag Summary

Figure 1 shows the failure-flag heatmap. Key patterns:

- **M8 (graph)** is failure-flagged on 100% of configs (25/25). M8 produces non-trivial anomaly scores (e.g., T1 AP=0.919, T3 AP=1.000) but fails to improve on the rule baseline on any configuration. The largest deficit is T6 (dormant reactivation): M8 AP=0.490 vs. M1 AP=1.000. Bipartite graph features capture structural position, not temporal account state.
- **M5 (OCSVM)** has the lowest failure rate (68.6%) and is not flagged on T2 or T5, the two tasks where ML adds value

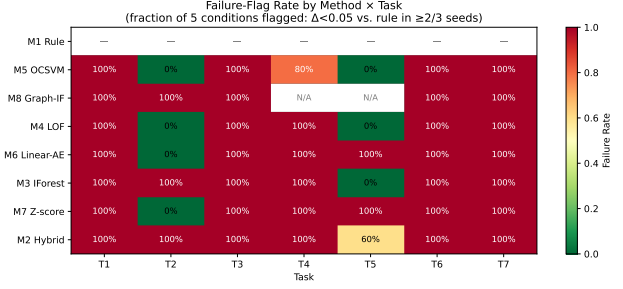


Fig. 1. Failure-flag rate heatmap (method $\times$ task). Red cells indicate flagged configurations, ML fails to improve on the rule baseline by the pre-registered $\Delta=0.05$ threshold (Eq. (1)) in $\geq 2/3$ seeds; white cells (M8, T4-T5) are not applicable.

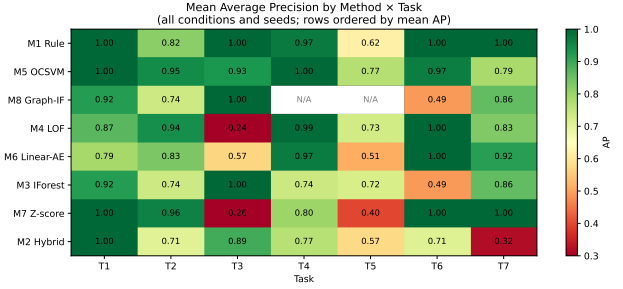


Fig. 2. Condition-averaged AP heatmap (method $\times$ task). Higher values indicate better anomaly ranking. Rows are ordered by mean AP across all runs; M1 (rule baseline) anchors the top row.

(it does fail Eq. (1) on T2 in occasional individual seeds, but never in $\geq 2/3$ seeds).

- **M2 (hybrid scorer)** is failure-flagged on 94.3% of configs, worst among non-graph methods. Its fixed-weight combination is dominated by the task-specific rule baseline.

VI. DISCUSSION

A. Interpretation of the Negative-Result Finding

The 83.8% failure rate is not a finding against ML in security operations broadly. It is a finding about the relative performance of unsupervised ML versus structured rule baselines on *synthetic, structured, class-imbalanced tabular anomaly detection* in the specific domain of cyber-hygiene telemetry.

The synthetic benchmark is, by design, favorable to rule-based methods: anomaly injection creates features that are directly correlated with the anomaly class in ways that rule thresholds exploit cleanly. In operational environments, data is noisier, labels are unavailable, and the feature space is more complex. HYGIENE BENCH reports this limitation explicitly in the dataset card and generator documentation.

The appropriate takeaway is: for HYGIENE BENCH v0.1, ML adds consistent value over rule baselines specifically on T2 (group membership drift) and T5 (patch/vulnerability hygiene). These tasks involve multi-signal combinations that benefit from learned representations. Future work should test whether this advantage persists under increased population-level noise and label sparsity.

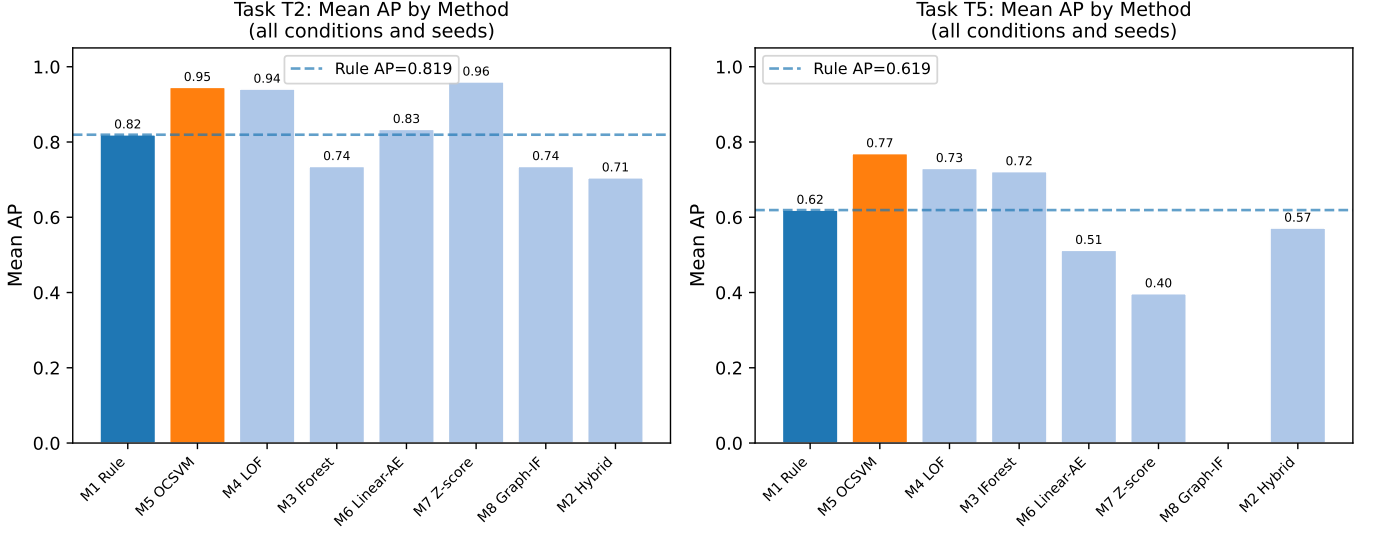


Fig. 3. AP by method for T2 (group membership drift) and T5 (patch-vulnerability hygiene), averaged over all conditions and seeds, the two tasks where ML consistently adds value over the rule baseline.

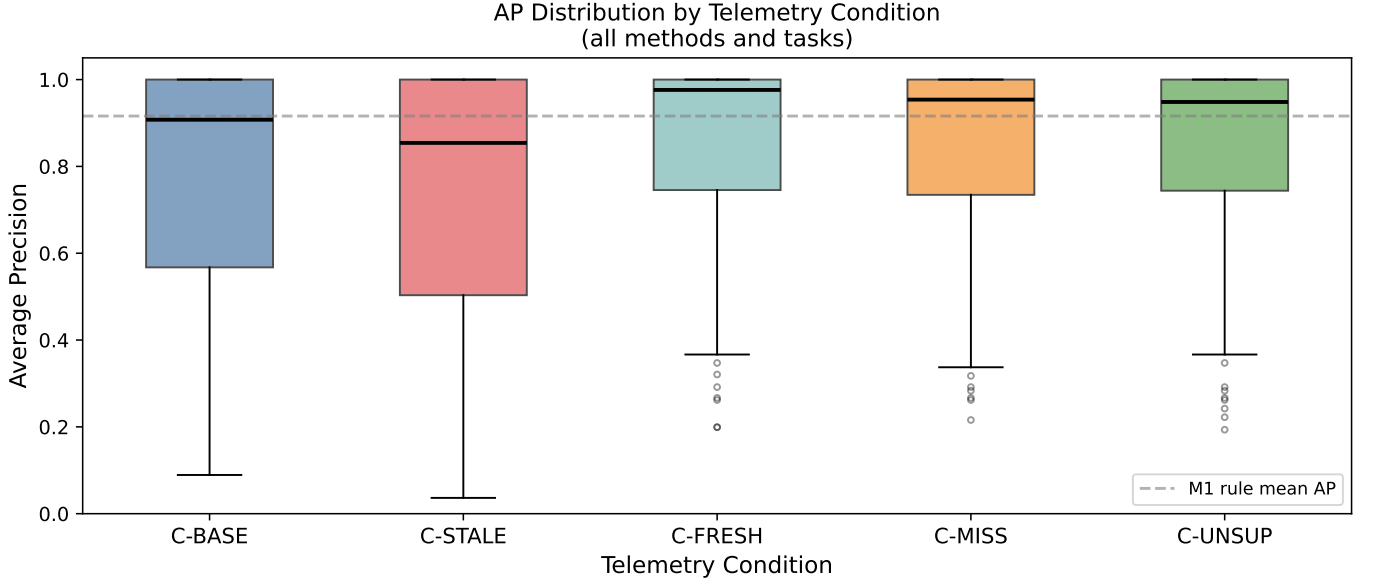


Fig. 4. AP distribution by telemetry condition (all tasks and methods). C-STALE has the lowest mean AP; C-FRESH has the highest.

B. Limitations

a) Synthetic data.: All telemetry is synthetic. Structural priors are approximate (DBIR 2026 [1] for patch lag; NVD for CVE severity; CISA BOD 23-01 for freshness cadence). AD group-size distributions lack a peer-reviewed source and are modeled under disclosed assumptions. Detection performance on real operational data will differ from benchmark results.

b) Approximations in M6 and M8.: M6 is implemented as PCA-based linear reconstruction error (not a neural autoencoder) and M8 as networkx graph features + Isolation Forest (not DOMINANT-style deep graph autoencoder [10]). These are disclosed approximations; results may change with full implementations.

c) Medium population scale.: The medium scale ($n=1,000$ users) limits the statistical power of the evaluation. The population is smaller than most public-sector environments, which may affect the generalizability of the class imbalance findings.

d) Feature engineering as a design choice.: The task feature extractors (T1-T7) encode domain knowledge about which features matter. Methods that exploit these features more effectively will appear better. A fully unsupervised method that discovers relevant features from raw tables is not evaluated.

e) Fixed k values.: $P@k$ is sensitive to the choice of k . HYGIENEBENCH uses operationally motivated k values (daily/weekly SOC review budgets) rather than tuning k to maximize method performance. This is a deliberate design

choice but limits comparison to benchmarks with different k conventions.

f) No calibration.: Anomaly scores are not calibrated to probabilities. Calibration is the primary research question of a separate line of work and is excluded from HYGIENE BENCH by design.

C. Relationship to Prior Work

HYGIENE BENCH is designed to complement, not replace, existing benchmarks. It does not claim to detect real network intrusions (LANL, CICIDS), insider threats (CERT), or attack-emulation events (Mordor, BOTSv3 [9]). It is the first benchmark, to our knowledge, that jointly covers Active Directory identity hygiene state, endpoint patch posture, vulnerability exposure, and telemetry freshness under controlled conditions with an openly released, reproducible synthetic generator.

VII. CONCLUSION

We introduced HYGIENE BENCH, a reproducible synthetic benchmark for cyber-hygiene anomaly detection, covering seven tasks and twelve anomaly classes across eleven entity/event tables. An 810-run evaluation of eight methods under five telemetry conditions finds that simple rule baselines perform at least as well as unsupervised ML on 83.8% of (condition, task, method) configurations. ML adds meaningful value on two of seven tasks, group membership drift (T2, +0.185 AP) and patch/vulnerability hygiene (T5, +0.210 AP), while rule baselines dominate on tasks with strong single-feature discriminators. Telemetry staleness (C-STALE) consistently degrades detection performance. These findings motivate honest, task-stratified evaluation of anomaly detection methods and explicit negative-result reporting in security benchmark papers.

HYGIENE BENCH v0.1, including the synthetic generator, frozen datasets, and evaluation harness, is available in an anonymized repository (link redacted for review; the repository and a Zenodo DOI will be released on acceptance).

REFERENCES

- [1] Verizon, “2026 Data Breach Investigations Report,” Verizon Business, 2026, <https://www.verizon.com/business/resources/reports/dbir/>.
- [2] CISA, “Binding Operational Directive 23-01: Improving Asset Visibility and Vulnerability Detection on Federal Networks,” Cybersecurity and Infrastructure Security Agency, Tech. Rep., Oct. 2022, <https://www.cisa.gov/binding-operational-directive-23-01>.
- [3] NIST, “SP 800-40 Rev. 4: Guide to Enterprise Patch Management Planning,” National Institute of Standards and Technology, Tech. Rep., Apr. 2022, <https://csrc.nist.gov/publications/detail/sp/800-40/rev-4/final>.
- [4] A. D. Kent, “Comprehensive, Multi-Source Cyber-Security Events,” Los Alamos National Laboratory, Tech. Rep., 2015, doi:10.2172/1259877.
- [5] J. Glasser and B. Lindauer, “Bridging the Gap: A Pragmatic Approach to Generating Insider Threat Data,” in *Proceedings of the IEEE Security and Privacy Workshops (SPW)*. San Francisco, CA, USA: IEEE, 2013, pp. 98-104.
- [6] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. Funchal, Madeira, Portugal: SciTePress, 2018, pp. 108-116.
- [7] R. M. Rodriguez, “Mordor: Pre-Recorded Security Events,” GitHub repository, 2019, <https://github.com/OTRF/Security-Datasets>.
- [8] DARPA Information Innovation Office (I2O), “Transparent Computing Engagement 3 (TC-E3) Data Release,” GitHub repository, Aug. 2018, <https://github.com/darpa-i2o/Transparent-Computing>.
- [9] Splunk, “Boss of the SOC (BOTS) v3 Dataset,” GitHub repository, 2018, <https://github.com/splunk/botsv3>.
- [10] K. Ding, J. Li, R. Bhanushali, and H. Liu, “Deep Anomaly Detection on Attributed Networks,” in *Proceedings of the 2019 SIAM International Conference on Data Mining (SDM)*. Calgary, AB, Canada: SIAM, 2019, pp. 594-602.
- [11] K. Liu, Y. Dou, X. Ding, X. Hu *et al.*, “PyGOD: A Python Library for Graph Outlier Detection,” *Journal of Machine Learning Research*, vol. 25, no. 141, pp. 1-9, 2024, <https://www.jmlr.org/papers/v25/23-0963.html>.
- [12] G. Creech and J. Hu, “Generation of a New IDS Test Dataset: Time to Retire the KDD Collection,” in *2013 IEEE Wireless Communications and Networking Conference (WCNC)*. Shanghai, China: IEEE, 2013, pp. 4487-4492.
- [13] D. Sculley, J. Snoek, A. Wiltschko, and A. Rahimi, “Winner’s Curse? On Pace, Progress, and Empirical Rigor,” in *Proceedings of the 6th International Conference on Learning Representations (ICLR), Workshop Track*. Vancouver, BC, Canada: OpenReview.net, 2018, pp. 1-4.